

PARADIGMAS DE PROGRAMACIÓN
CURSO SUAREZ

Trabajo Práctico 1 Programación Orientada a Objetos

× ×

24 de septiembre de 2026

Alexander Cruz
109044
acruz@fi.uba.ar

Índice

1. Introducción	3
2. Objetivo	3
3. Consigna	3
4. Alcance	3
5. Detalles de Implementación	4
5.1. Ejemplos de Polimorfismo	4
5.1.1. Jerarquía de Artista	4
5.1.2. Jerarquía de Ubicación	5
5.1.3. Jerarquía de Medio de Pago	6
5.2. Mecanismos Utilizados	6
6. Diagrama de Clases	7
7. Diagramas de Secuencia	8
7.1. Caso de Uso 1: Registro de un Evento	8
7.2. Caso de Uso 2: Cálculo del Costo de una Entrada	8
7.3. Caso de Uso 3: Obtención de la Entrada Más Barata	9
8. Cobertura	10

1. Introducción

El presente informe detalla la solución desarrollada para el Trabajo Práctico 1, que tiene como objetivo aplicar los conceptos fundamentales de la programación orientada a objetos (POO) en el lenguaje Pharo. El problema, que consiste en el desarrollo de un sistema de recomendación de futbolistas, fue resuelto utilizando la metodología Test-Driven Development (TDD), un enfoque que garantiza la robustez y funcionalidad del código a través de pruebas unitarias.

La solución se centra en la construcción de un modelo de clases que se adhiere a los principios de la POO, como el encapsulamiento y, de manera crucial, el polimorfismo. Se ha diseñado una arquitectura que permite manejar la lógica de cálculo del valor de los futbolistas y los distintos criterios de búsqueda de manera flexible y extensible, evitando el uso de estructuras de control rígidas.

Adicionalmente, se ha utilizado la herramienta Mutalk para llevar a cabo una prueba de mutación. Este proceso aseguró la calidad de las pruebas unitarias creadas por el alumno, garantizando que el código no solo funciona, sino que está correctamente validado contra posibles errores. El informe incluye los diagramas de clases y de secuencia que documentan el diseño, junto con una explicación detallada de la implementación y los mecanismos de polimorfismo utilizados.

2. Objetivo

Este es un trabajo práctico individual que tiene como objetivo aplicar los conceptos del paradigma de la orientación a objetos vistos hasta ahora en el curso mediante la resolución de un problema con Pharo.

3. Consigna

Se deberá desarrollar un modelo de clases utilizando la metodología TDD. Las especificaciones son un archivo de pruebas SUnit (.st) adjunto a este enunciado y están dadas en forma de pruebas de casos de uso. Se espera que el alumno genere una solución que permita ejecutar exitosamente las pruebas especificadas manteniendo una alta cobertura y evitando la supervivencia de mutantes. Revisar el material Calidad de nuestras pruebas.

4. Alcance

Se deberá desarrollar el modelo completo de la aplicación, sin incluir la interfaz gráfica. Para esto se especifica una API que está descrita por un conjunto de casos de uso. Este conjunto de pruebas está descrito por la clase dada como enunciado.

Para el manejo de errores y situaciones de excepción, se deberán desarrollar y utilizar las excepciones que se crean convenientes, más allá de que en la prueba de integración provistas no necesariamente se manejan excepciones.

Deberán entregarse:

- todas las clases con sus métodos que resuelven el problema, organizados en una o más categorías según criterio del alumno;
- conjunto de pruebas que garanticen la no supervivencia de mutantes;
- conjunto de pruebas que garanticen la no supervivencia de mutantes;
- un archivo ".txt" que contenga la pieza de código que permite ejecutar los test de mutación (descrito aquí: pruebas de mutación);

documentación completa del diseño de clases incluyendo diagramas UML de clases y secuencia (cantidades variables acordes al diseño presentado). Se considerarán entregas válidas a aquellas que contengan las partes anteriores. Las que no tengan alguna de ellas no recibirán corrección.

5. Detalles de Implementación

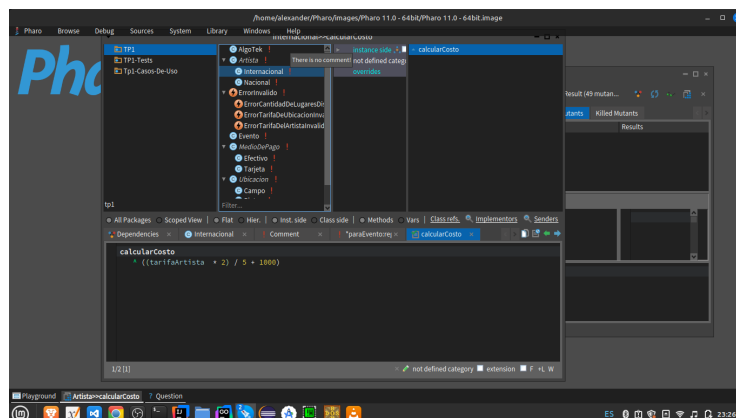
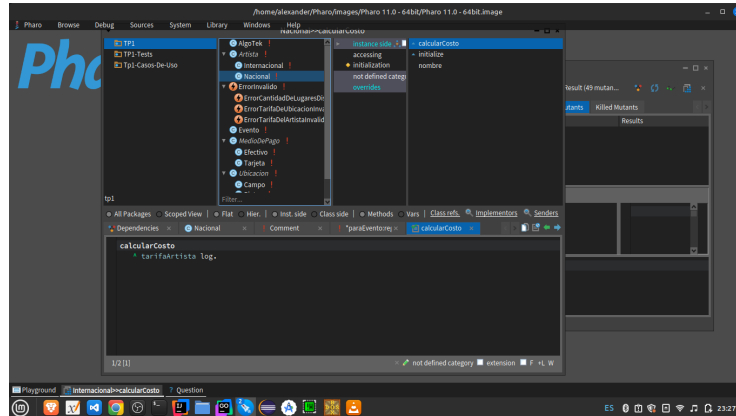
5.1. Ejemplos de Polimorfismo

El ****polimorfismo**** es uno de los pilares fundamentales de la Programación Orientada a Objetos. En el sistema **AlgoTek**, este concepto permite tratar distintos objetos de manera uniforme a través de una interfaz o conjunto de mensajes compartidos, desacoplando a los clientes de las especificaciones concretas de cada subclase.

5.1.1. Jerarquía de Artista

El primer ejemplo claro de polimorfismo se encuentra en las clases **Nacional** e **Internacional**, las cuales heredan de la clase abstracta **Artista**. Cada subclase implementa su propio cálculo de costos mediante el método **calcularCosto**:

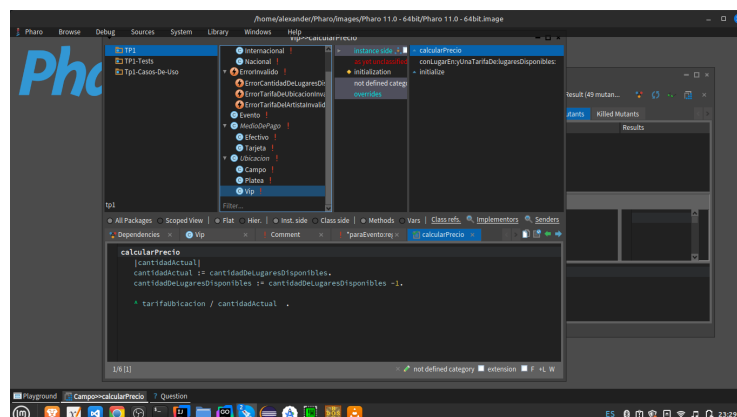
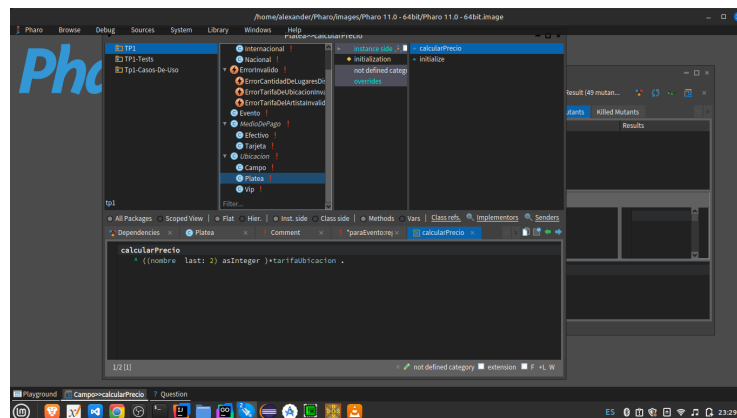
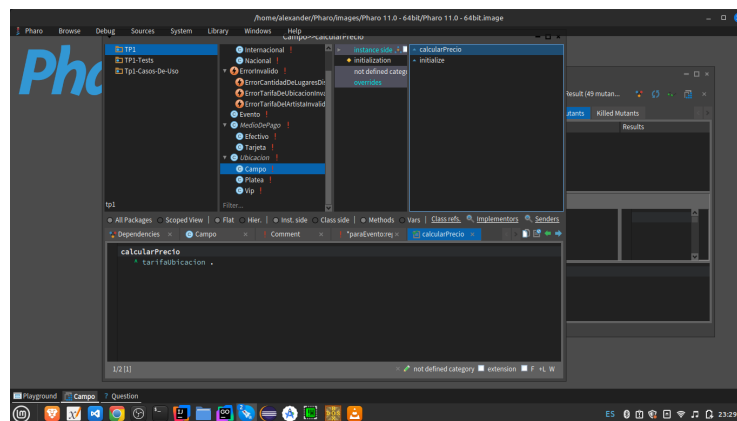
- La clase **Nacional** calcula su costo aplicando un cálculo logarítmico sobre su tarifa base ($\text{tarifaArtista} \log$).
- La clase **Internacional** utiliza una fórmula con un recargo adicional sobre su tarifa base ($((\text{tarifaArtista} * 2) / 5 + 1000)$).



5.1.2. Jerarquía de Ubicación

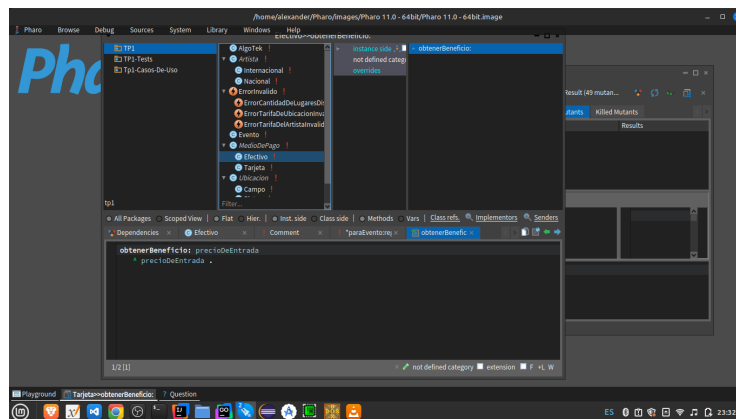
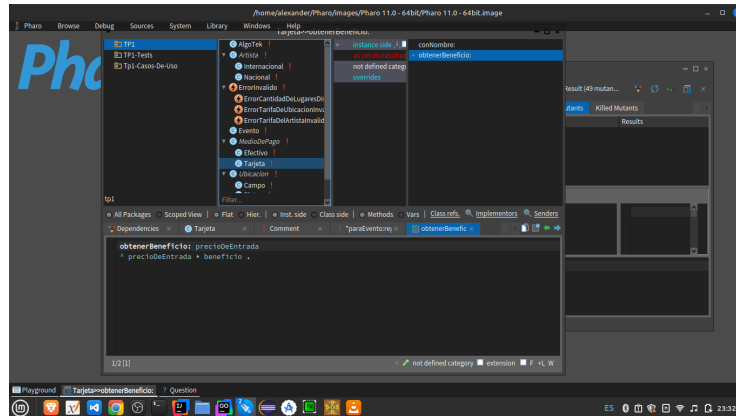
Como segundo ejemplo de polimorfismo, encontramos la jerarquía de **Ubicacion**, conformada por las subclases **Campo**, **Platea** y **Vip**. Cada una responde al mensaje `calcularPrecio` de forma totalmente autónoma según su lógica de negocio:

- **Campo**: Retorna directamente la tarifa fija asignada.
- **Platea**: Determina su valor multiplicando el número de la fila/asiento por la tarifa de la ubicación.
- **Vip**: Modifica dinámicamente su precio según la disponibilidad actual de lugares (`tarifaUbicacion / cantidadActual`) y actualiza los lugares disponibles.



5.1.3. Jerarquía de Medio de Pago

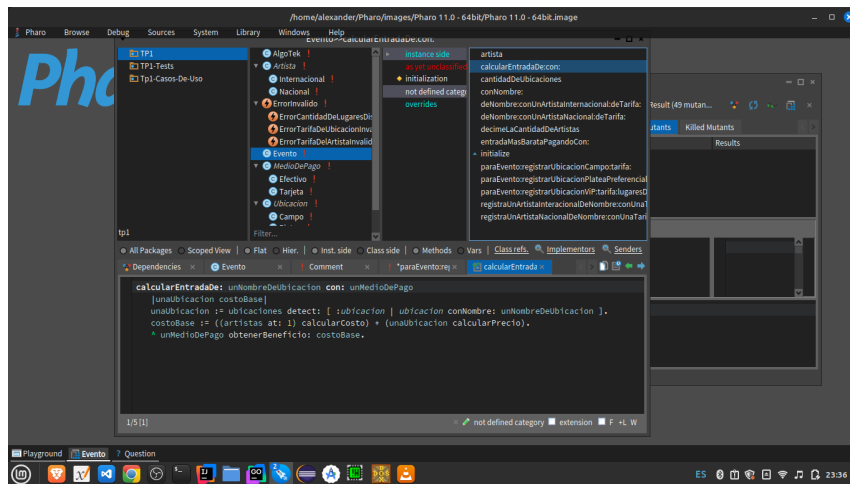
Un tercer ejemplo es el cálculo de descuentos aplicados a las entradas mediante la jerarquía de MedioDePago (Efectivo y Tarjeta), compartiendo el mensaje polimórfico `obtenerBeneficio: precioDeEntrada`.



5.2. Mecanismos Utilizados

El objeto `Evento`, actuando como cliente, interactúa con la colección de ubicaciones y artistas invocando los mensajes `calcularPrecio` y `calcularCosto` sin conocer los detalles de implementación de cada subclase concreta.

De igual manera, el objeto `AlgoTek` puede delegar el cálculo final del beneficio invocando `obtenerBeneficio:` sobre cualquier instancia de `MedioDePago`. Este mecanismo promueve un sólido **encapsulamiento** y la aplicación del principio de responsabilidad única (SRP), permitiendo extender el sistema agregando nuevos tipos de artistas, ubicaciones o medios de pago sin modificar el código del cliente.



6. Diagrama de Clases

El diagrama representa la arquitectura del sistema **AlgoTek** diseñado para la gestión de eventos masivos, venta de entradas y cálculo de tarifas. El diseño se centra en la flexibilidad, el encapsulamiento y el aprovechamiento del polimorfismo para manejar distintos tipos de artistas, tipos de ubicación y medios de pago.

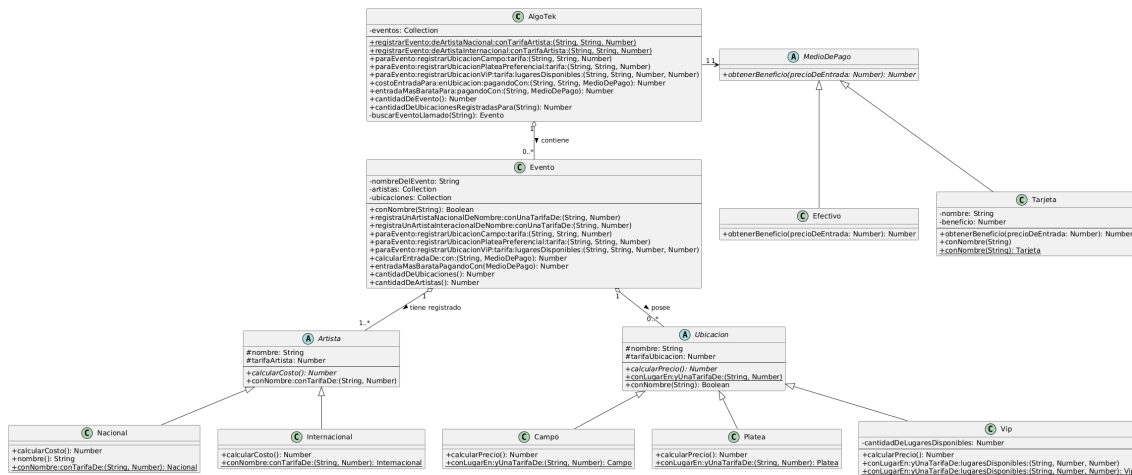
Estructura de Clases y Jerarquía

- **AlgoTek:** Clase principal del sistema que actúa como punto de entrada (Facade). Mantiene una colección de instancias de **Evento** y se encarga de delegar el registro de ubicaciones, cálculo de costos de entradas y búsqueda de ofertas.
- **Evento:** Representa un evento en el sistema. Contiene una colección de objetos que heredan de **Artista** y una colección de objetos que heredan de **Ubicacion**. Coordina el cálculo del costo base de las entradas delegando la lógica en las entidades correspondientes.
- **Jerarquía de Artista (Artista, Nacional, Internacional):** Define una clase abstracta **Artista** con la interfaz común **calcularCosto**. Las subclases concretas encapsulan sus propios algoritmos de costos base según la categoría del artista.
- **Jerarquía de Ubicación (Ubicacion, Campo, Platea, Vip):** Define la estructura y precio de los distintos sectores de un evento. Cada subclase implementa de forma autónoma el método **calcularPrecio** (por ejemplo, **Vip** ajusta dinámicamente su valor según la disponibilidad de lugares).
- **Jerarquía de Medio de Pago (MedioDePago, Efectivo, Tarjeta):** Encapsula la lógica de descuentos y beneficios sobre los precios finales mediante el método polimórfico **obtenerBeneficio::**

Relaciones y Comportamientos

- **Polimorfismo:** Se observa principalmente en los métodos **calcularCosto** (en **Artista**), **calcularPrecio** (en **Ubicacion**) y **obtenerBeneficio::** (en **MedioDePago**). Esto permite que **Evento** y **AlgoTek** interactúen con colecciones heterogéneas sin necesidad de consultar el tipo concreto de cada objeto.
- **Agregación y Composición:** **AlgoTek** mantiene una relación de agregación con una colección de objetos **Evento** (0..*). A su vez, cada **Evento** se compone de colecciones de **Artista** (1..*) y **Ubicacion** (0..*).

- **Manejo de Excepciones:** El sistema presenta dependencias hacia la jerarquía de excepciones (`ErrorTarifaDelArtistaInvalido`, `ErrorTarifaDeUbicacionInvalido`, `ErrorCantidadDeLugaresDispon` entre otras). Estas son señaladas durante la instanciación de los objetos para garantizar la consistencia de los datos del dominio.

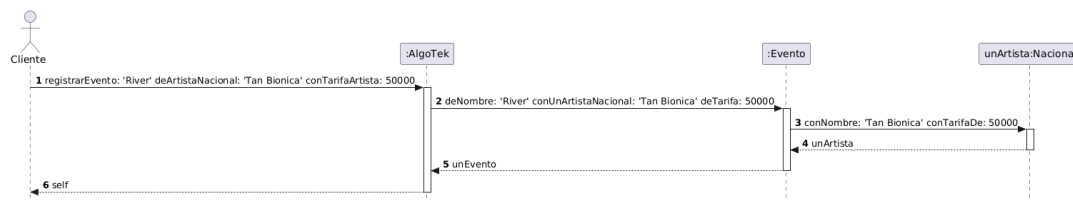


7. Diagramas de Secuencia

Los diagramas de secuencia ilustran la interacción temporal entre las distintas instancias del sistema ante la ejecución de los casos de uso principales. A través de estos diagramas se observa cómo se aplica el envío de mensajes en Smalltalk, la delegación de responsabilidades y la resolución polimórfica en tiempo de ejecución.

7.1. Caso de Uso 1: Registro de un Evento

El cliente solicita a **AlgoTek** registrar un nuevo evento indicando si el artista es nacional o internacional y su tarifa inicial. La clase de dominio **AlgoTek** instancia el objeto **Evento**, el cual a su vez crea e incorpora a su colección interna la subclase concreta de **Artista** (en este caso, **Nacional**).



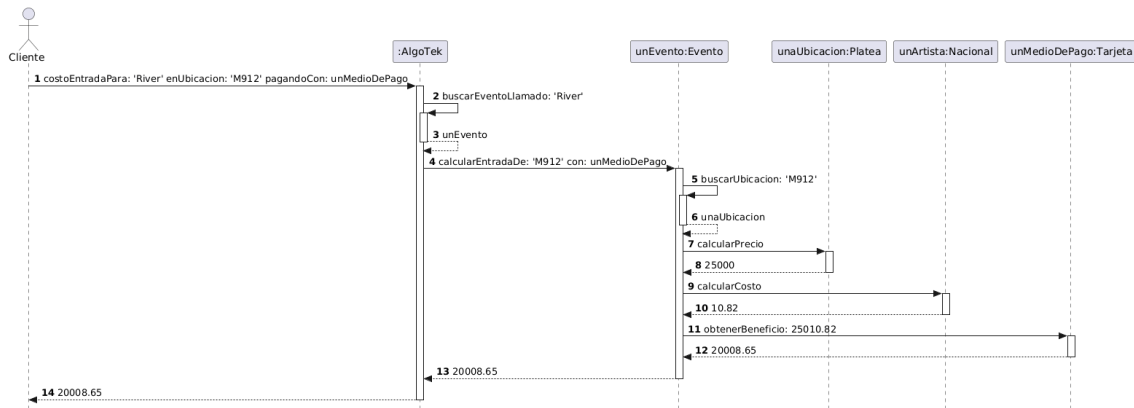
7.2. Caso de Uso 2: Cálculo del Costo de una Entrada

Este diagrama muestra el flujo al consultar el costo final de una entrada específica mediante `costoEntradaPara:enUbicacion:pagandoCon:.`

Se aprecia claramente la delegación de responsabilidades:

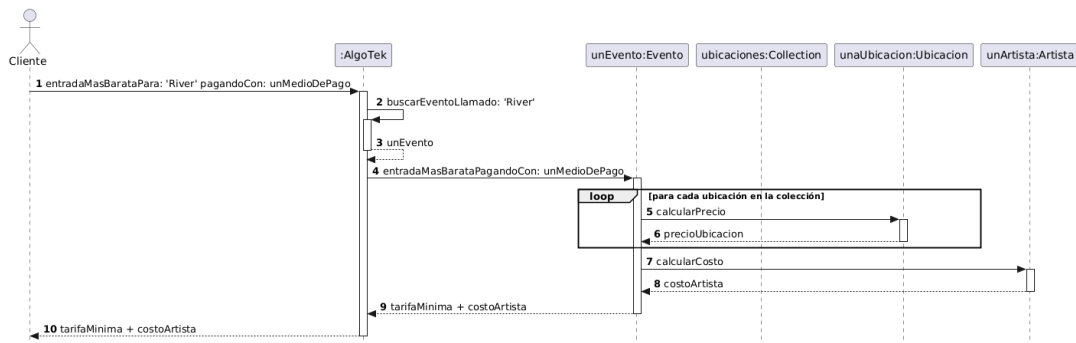
1. **AlgoTek** busca el objeto **Evento** correspondiente y le delega el cálculo.
2. **Evento** obtiene la ubicación requerida y le envía el mensaje polimórfico **calcularPrecio**.
3. **Evento** solicita a su **Artista** el costo base enviándole **calcularCosto**.

4. Finalmente, **Evento** envía el mensaje **obtenerBeneficio:** a la instancia de **MedioDePago** pasando la suma de los costos base para aplicar el descuento correspondiente.



7.3. Caso de Uso 3: Obtención de la Entrada Más Barata

Para obtener la entrada más económica de un evento, el cliente envía el mensaje **entradaMasBarataPara:pagandoCo**. El objeto **Evento** itera sobre su colección de ubicaciones enviando el mensaje polimórfico **calcularPrecio** a cada una, determina la tarifa mínima y le suma el costo del artista antes de retornar el total.



8. Cobertura

